Azure - Part 1

6-MONTH INDUSTRY MASTERCLASS PATHWAY

An enterprise-grade, comprehensive technical architecture, deployment workflow, security hardening guide, and observability plan. Designed for senior engineering professionals (6+ years experience) striving for Principal SRE and Cloud Architect roles.

CORE CONCEPTS | PRODUCTION HARDENING | 20+ INCIDENT SCENARIOS

Automated SRE Learn System | Generated Pdf Generator

Azure DevOps & Cloud Architecture Study Guide

Part 1/3: Core Foundations, Topologies, and Identity

---

1. Part Introduction and Scope

This study guide is designed for senior IT professionals (6+ years of experience) aiming to master Azure Cloud Architecture and DevOps engineering within a 6-month timeline.

Part 1 covers the foundational pillars of Microsoft Azure:

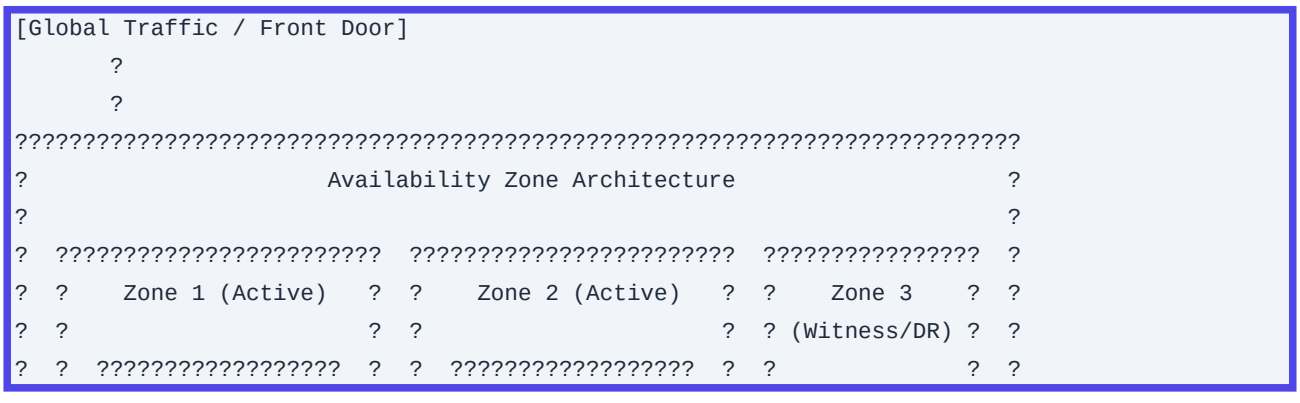
- **Azure Resource Manager (ARM) Engine:** Declarative deployment, resource providers, and subscription/management group topologies.
- **Enterprise Networking Foundations:** Hub-Spoke virtual networks, Azure Virtual WAN (vWAN), Private Link/Private Endpoints, Azure Firewall Premium, and User-Defined Routing (UDR).
- **Identity & Access Management (IAM):** Microsoft Entra ID, Managed Identities (System/User-assigned), Service Principals, and Role-Based/Attribute-Based Access Control (RBAC/ABAC).
- **Compute Foundations:** Virtual Machine Scale Sets (VMSS), Availability Zones, and Accelerated Networking.

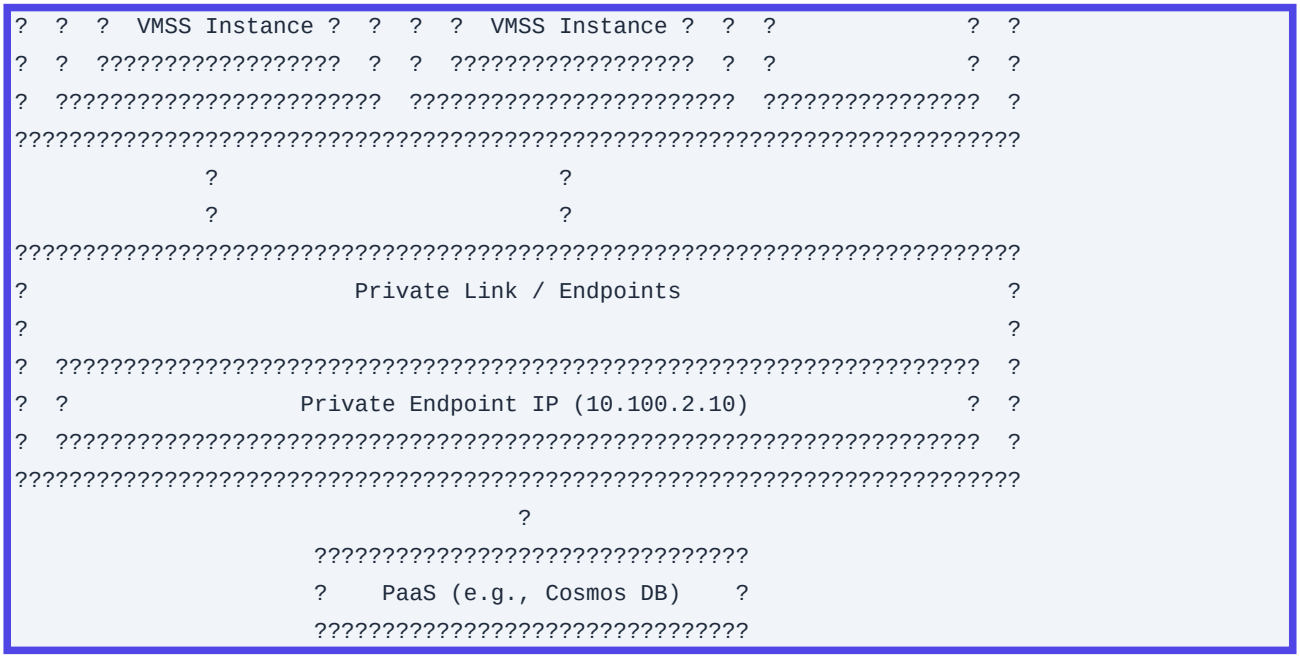
By mastering these core elements, you will gain the architectural knowledge required to design, secure, and automate enterprise-grade landing zones.

---

2. Architectural Importance for High-Availability Systems

Enterprise systems demand resilience against infrastructure failures, network partitions, and security breaches. Understanding Azure's foundational layer is critical for high availability (HA) for several key reasons:





Blast Radius Isolation

Subscriptions act as logical boundaries for billing and security, but more importantly, they define ARM API rate limits.

By designing a multi-subscription topology (e.g., separating production, non-production, and shared services), you prevent a runaway deployment script in dev from exhausting ARM API quotas and blocking critical production scaling operations.

Control Plane vs. Data Plane Resilience

- **Control Plane (ARM):** Responsible for management tasks (e.g., creating resources). It operates on an asynchronous, eventually consistent model.
- **Data Plane (e.g., VNet Routing, Virtual Machines):** Operates independently. If the ARM control plane experiences a regional outage, existing VNets, VMSS instances, and Private Endpoints continue to route traffic and serve requests without interruption.

High-Availability Networking

Traditional active-passive network appliances introduce single points of failure and routing bottlenecks.

By using native, software-defined services like Azure Firewall Premium (configured across multiple Availability Zones) and Azure Private Link, you ensure that network paths scale elastically and tolerate physical datacenter failures.

---

3. Real-World Enterprise Use Cases

Use Case 1: Secure Enterprise Landing Zone (Hub-Spoke Topology)

- **Context:** A global financial services institution needs to migrate legacy workloads to Azure while maintaining strict regulatory compliance (PCI-DSS, SOC2).
- **Requirements:**
- Zero direct internet access for workload VNets.
- Centralized egress inspection via next-generation firewalls.
- Private, non-transitive network paths between applications.
- **Solution:** Implement a Hub-Spoke topology using Azure Firewall Premium in the Hub VNet. All Spoke VNets route 100% of outbound traffic (`0.0.0.0/0`) through the Hub Firewall using User-Defined Routes (UDRs).
- **Data Access:** Spoke resources access multi-tenant PaaS services (e.g., Azure SQL, Key Vault) exclusively via Private Endpoints placed within dedicated subnets inside the spokes, preventing data exfiltration to external public endpoints.

Use Case 2: Multi-Region Active-Active API Platform

- **Context:** An e-commerce platform requires sub-second response times globally and must tolerate complete regional outages.
- **Requirements:**
- Automated compute scaling based on real-time traffic.
- Zero-trust communication between APIs and stateful databases.
- Zero-downtime deployments.
- **Solution:** Deploy Virtual Machine Scale Sets (VMSS) across three Availability Zones in two regions (e.g., East US 2 and West US 3).
- **Scaling & Security:** Configure VMSS with Accelerated Networking (SR-IOV) enabled to reduce latency. Secure the VMs by assigning a User-Assigned Managed Identity, allowing them to retrieve secrets from Azure Key Vault via Private Link without using hardcoded credentials.
- **Routing:** Use Azure Front Door to route traffic globally, with Azure Application Gateway v2 acting as the local regional ingress and Web Application Firewall (WAF).

---

4. Comprehensive Architecture Explanation

The Hub-Spoke architecture is the foundation of secure enterprise deployments in Azure. Below is a detailed view of a production-ready Hub-Spoke topology integrated with Microsoft Entra ID and Private Link.

```
graph TB
    subgraph Azure Tenant [Microsoft Entra ID Tenant]
```

```
        subgraph Management Group: Platform [Platform Management Group]
            subgraph Sub: Connectivity [Connectivity Subscription]
                subgraph VNet: Hub [Hub VNet - 10.100.0.0/16]
                    subgraph Subnet: Gateway [GatewaySubnet]
                        VPNGW[ExpressRoute / VPN Gateway]
                    end
                    subgraph Subnet: Firewall [AzureFirewallSubnet]
                        AzFW[Azure Firewall Premium]
                    end
                    subgraph Subnet: Bastion [AzureBastionSubnet]
                        Bastion[Azure Bastion]
                    end
                end
            end
        end

        subgraph Management Group: Landing Zones [Landing Zones Management Group]
            subgraph Sub: Production [Production Subscription]
                subgraph VNet: Spoke A [Spoke A VNet - 10.101.0.0/16]
                    subgraph Subnet: Web [Web Subnet - 10.101.1.0/24]
                        VMSS[Virtual Machine Scale Set]
                    end
                    subgraph Subnet: PrivateEndpoints [PE Subnet - 10.101.2.0/24]
                        PE_KV[Private Endpoint: Key Vault]
                    end
                end
            end
        end

        subgraph Azure PaaS Services [Azure PaaS Platform]
            AKV[Azure Key Vault]
        end
    end

    %% Network Peerings
    VNet: Hub <-->|VNet Peering <br> AllowGatewayTransit| VNet: Spoke A

    %% Routing Paths
    VMSS -->|UDR: 0.0.0.0/0 -> AzFW IP| AzFW
    VMSS -->|Private Link Access| PE_KV
    PE_KV -.->|Private Link Tunnel| AKV

    %% On-Premises Connectivity
    OnPrem[On-Premises Datacenter] <-->|ExpressRoute Circuit| VPNGW

    %% Styles
    style Azure Tenant fill:#f9f9f9,stroke:#333,stroke-width:2px
    style VNet: Hub fill:#e1f5fe,stroke:#0288d1,stroke-width:2px
```

```
    style VNet: Spoke A fill:#e8f5e9,stroke:#388e3c,stroke-width:2px
    style Azure PaaS Services fill:#fff3e0,stroke:#f57c00,stroke-width:2px
```

Structural Components

1. Hub VNet (`10.100.0.0/16`)

The central point of connectivity for the cloud footprint. It hosts shared infrastructure services.

- **AzureFirewallSubnet (`10.100.1.0/24`):** Hosts Azure Firewall Premium. This subnet must be named exactly `AzureFirewallSubnet` and should be at least a `/26` to allow for scaling.
- **GatewaySubnet (`10.100.2.0/24`):** Hosts the ExpressRoute or VPN Gateway used for hybrid connectivity to on-premises networks. This subnet must be named exactly `GatewaySubnet`.
- **AzureBastionSubnet (`10.100.3.0/26`):** Hosts Azure Bastion, providing secure, browser-based RDP/SSH access to VMs without exposing public IPs.

2. Spoke VNet (`10.101.0.0/16`)

Isolated network boundaries where application workloads reside.

- **Web Subnet (`10.101.1.0/24`):** Hosts compute resources like VMSS.
- **Private Endpoint Subnet (`10.101.2.0/24`):** Dedicated subnet for Private Endpoints. Placing endpoints here secures access to PaaS services.

3. Global VNet Peering

Connects the Hub and Spoke VNets with low latency using Microsoft's backbone network.

- **Use Remote Gateways (Spoke side):** Allows workloads in the spoke to use the VPN/ExpressRoute gateway in the Hub.
- **Allow Gateway Transit (Hub side):** Permits the Hub gateway to route traffic for peered spokes.

4. Private Link and DNS Integration

- When a Private Endpoint is created for Azure Key Vault, it is assigned a private IP from `10.101.2.0/24`.
- A Private DNS Zone (`privatelink.vaultcore.azure.net`) is linked to the Spoke VNet. This ensures that queries for `myvault.vault.azure.net` resolve to the private IP (`10.101.2.x`) instead of its public IP.

---

5. Component Classifications and Deep Dives

Identity & Access Management (IAM)

1. Service Principals vs. Managed Identities

| Feature | Service Principal (App Registration) | System-Assigned Managed Identity | User-Assigned Managed Identity |

| :--- | :--- | :--- | :--- |

| Lifecycle | Independent of any Azure resource. Managed manually or via CI/CD. | Tied directly to the lifecycle of a single Azure resource. | Independent lifecycle; can be shared across multiple resources. |

| Credential Management | Manual (Certificates or Client Secrets with expiration dates). | Automated by Azure. No credentials exposed to developers. | Automated by Azure. No credentials exposed to developers. |

| Use Case | External CI/CD pipelines (e.g., GitHub Actions, Jenkins), external applications. | Single-resource workloads (e.g., a dedicated VM running a single service). | Multi-resource workloads (e.g., a scale set of VMs that all need the same permissions). |

2. RBAC vs. ABAC

- **RBAC (Role-Based Access Control):** Grants permissions based on roles (e.g., *Contributor*, *Reader*, *User Access Administrator*) assigned at a specific scope (Management Group, Subscription, Resource Group, or Resource).
- **ABAC (Attribute-Based Access Control):** Builds on RBAC by adding conditions based on attributes. For example, you can grant a user the *Storage Blob Data Reader* role, but restrict access to blobs that have a specific tag (e.g., `Project=Phoenix`).

Networking Foundations

1. VNet Peering: Regional vs. Global

- **Regional Peering:** Connects VNets within the same Azure region. Traffic travels over local high-speed physical fabrics.
- **Global Peering:** Connects VNets across different physical Azure regions. Traffic travels across Microsoft's global private fiber backbone, bypassing the public internet.

2. Private Endpoints vs. Service Endpoints

```
[Service Endpoint]
VM Subnet (10.100.1.0/24) ???? Public IP (A.B.C.D) ???? Azure SQL (Multi-tenant Firewalled
Public IP)
                                                    * No Private IP in VNet
                                                    * Risk of data exfiltration to other
SQL instances

[Private Endpoint]
VM Subnet (10.100.1.0/24) ???? Private IP (10.100.1.5) ???? Private Link ???? Dedicated Azure
```

```
SQL Instance
                                                        * Traffic never leaves VNet
address space
                                                        * Access to other instances
blocked by default
```

- **Service Endpoints:**
- Keeps traffic on the Azure backbone but routes to the PaaS service's *public* IP.
- The PaaS service's firewall is configured to allow traffic from the VNet's subnet.
- *Risk:* Does not protect against data exfiltration, as compromised workloads can still write to external, unauthorized PaaS instances of the same service type.
- **Private Endpoints:**
- Allocates a dedicated *private* IP address from your VNet's CIDR block to the PaaS service.
- Traffic is routed over a private link directly to the specific resource.
- *Security:* Eliminates data exfiltration risks by completely disabling public network access to the PaaS resource.

Compute Foundations

1. Virtual Machine Scale Sets (VMSS)

VMSS allows you to deploy and manage a group of load-balanced VMs. It supports two orchestration modes:

- **Uniform Orchestration:** Uses a single VM profile to deploy identical instances. Best for large-scale, stateless workloads.
- **Flexible Orchestration (VMSS Flex):** Allows you to mix spot and on-demand VMs, choose different VM sizes, and manage individual VMs within the group. This provides high availability across fault domains without requiring custom deployment scripts.

2. Availability Options

```
[Availability Set]
Rack 1 (FD 0, UD 0) ??? VM01  \  Protects against local hardware/rack failures
Rack 2 (FD 1, UD 1) ??? VM02  /  (Single Datacenter)

[Availability Zones]
Zone 1 (Datacenter A) ??? VM01 \  Protects against complete datacenter outages
Zone 2 (Datacenter B) ??? VM02 /  (Fiber-connected, distinct power/cooling)
```

- **Availability Sets:** Groups VMs within a single datacenter, distributing them across Fault Domains (physical racks sharing power/cooling) and Update Domains (logical groups not rebooted simultaneously during maintenance).

- **Availability Zones (AZs):** Physically separate datacenters within an Azure region. Each zone has independent power, cooling, and networking. AZs provide the highest SLA for VM availability (99.99% for multi-zone VMs).

---

6. Step-by-Step Production Implementation Guide

This guide walks you through deploying a secure, Hub-Spoke network topology with Azure Firewall, Private DNS integration, and a secured Spoke VM using a User-Assigned Managed Identity.

Prerequisites

- Azure CLI installed and authenticated (`az login`).
- Active Azure Subscription with Owner permissions.

Phase 1: Set Up Resource Group and Hub Network

```
# 1. Define variables
export RG_NAME="rg-enterprise-core-prod-001"
export LOCATION="eastus2"
export HUB_VNET_NAME="vnet-hub-prod-001"
export SPOKE_VNET_NAME="vnet-spoke-prod-001"

# 2. Create the Resource Group
az group create --name $RG_NAME --location $LOCATION

# 3. Create the Hub Virtual Network
az network vnet create \
  --resource-group $RG_NAME \
  --name $HUB_VNET_NAME \
  --address-prefixes 10.100.0.0/16 \
  --subnet-name SharedServicesSubnet \
  --subnet-prefixes 10.100.1.0/24

# 4. Create dedicated subnets for Azure Firewall and Bastion in the Hub VNet
az network vnet subnet create \
  --resource-group $RG_NAME \
  --vnet-name $HUB_VNET_NAME \
  --name AzureFirewallSubnet \
  --address-prefixes 10.100.2.0/24

az network vnet subnet create \
  --resource-group $RG_NAME \
  --vnet-name $HUB_VNET_NAME \
  --name AzureBastionSubnet \
```

```
  --address-prefixes 10.100.3.0/26
```

Phase 2: Set Up Spoke Network and Peering

```
# 1. Create the Spoke VNet
az network vnet create \
  --resource-group $RG_NAME \
  --name $SPOKE_VNET_NAME \
  --address-prefixes 10.101.0.0/16 \
  --subnet-name WebSubnet \
  --subnet-prefixes 10.101.1.0/24

# 2. Create a dedicated subnet for Private Endpoints in the Spoke VNet
az network vnet subnet create \
  --resource-group $RG_NAME \
  --vnet-name $SPOKE_VNET_NAME \
  --name PrivateEndpointSubnet \
  --address-prefixes 10.101.2.0/24

# 3. Peer Hub VNet to Spoke VNet
az network vnet peering create \
  --name HubToSpoke \
  --resource-group $RG_NAME \
  --vnet-name $HUB_VNET_NAME \
  --remote-vnet $SPOKE_VNET_NAME \
  --allow-vnet-access

# 4. Peer Spoke VNet to Hub VNet
az network vnet peering create \
  --name SpokeToHub \
  --resource-group $RG_NAME \
  --vnet-name $SPOKE_VNET_NAME \
  --remote-vnet $HUB_VNET_NAME \
  --allow-vnet-access
```

Phase 3: Deploy Azure Firewall Premium

```
# 1. Create a Public IP for Azure Firewall
az network public-ip create \
  --resource-group $RG_NAME \
  --name pip-firewall-prod-001 \
  --location $LOCATION \
  --allocation-method Static \
  --sku Standard

# 2. Deploy Azure Firewall Policy (Premium)
az network firewall policy create \
```

```
  --resource-group $RG_NAME \
  --name afwp-core-prod-001 \
  --sku Premium \
  --location $LOCATION

# 3. Create the Azure Firewall instance
az network firewall create \
  --resource-group $RG_NAME \
  --name afw-hub-prod-001 \
  --location $LOCATION \
  --sku Premium \
  --firewall-policy afwp-core-prod-001

# 4. Configure IP Configurations for the Firewall
az network firewall ip-config create \
  --resource-group $RG_NAME \
  --firewall-name afw-hub-prod-001 \
  --name ipconfig-fw-prod-001 \
  --public-ip pip-firewall-prod-001 \
  --vnet-name $HUB_VNET_NAME
```

Phase 4: Configure User-Defined Routes (UDR)

```
# 1. Retrieve the private IP of Azure Firewall
FW_PRIVATE_IP=$(az network firewall show \
  --resource-group $RG_NAME \
  --name afw-hub-prod-001 \
  --query "ipConfigurations[0].privateIpAddress" \
  --output tsv)

# 2. Create a Route Table for the Spoke VNet
az network route-table create \
  --resource-group $RG_NAME \
  --name rt-spoke-web-prod-001 \
  --location $LOCATION

# 3. Create a default route (0.0.0.0/0) directing traffic to Azure Firewall
az network route-table route create \
  --resource-group $RG_NAME \
  --route-table-name rt-spoke-web-prod-001 \
  --name DefaultEgressToFirewall \
  --address-prefix 0.0.0.0/0 \
  --next-hop-type VirtualAppliance \
  --next-hop-ip-address $FW_PRIVATE_IP

# 4. Associate the Route Table with the Spoke Web Subnet
az network vnet subnet update \
  --resource-group $RG_NAME \
```

```
  --vnet-name $SPOKE_VNET_NAME \
  --name WebSubnet \
  --route-table rt-spoke-web-prod-001
```

Phase 5: Establish Identity and Access Control

```
# 1. Create a User-Assigned Managed Identity
az identity create \
  --resource-group $RG_NAME \
  --name id-spoke-app-prod-001 \
  --location $LOCATION

# 2. Retrieve the Principal ID of the Managed Identity
IDENTITY_PRINCIPAL_ID=$(az identity show \
  --resource-group $RG_NAME \
  --name id-spoke-app-prod-001 \
  --query "principalId" \
  --output tsv)

# 3. Assign the "Reader" role to the Managed Identity at the Resource Group scope
az role assignment create \
  --assignee $IDENTITY_PRINCIPAL_ID \
  --role "Reader" \
  --scope "/subscriptions/$(az account show --query id -o tsv)/resourceGroups/$RG_NAME"
```

---

7. Deep-Dive CLI Command Reference

1. `az network vnet create`

Creates a software-defined Virtual Network.

```
az network vnet create \
  --resource-group rg-enterprise-core-prod-001 \
  --name vnet-spoke-prod-001 \
  --address-prefixes 10.101.0.0/16 \
  --subnet-name WebSubnet \
  --subnet-prefixes 10.101.1.0/24
```

- `--address-prefixes`: Defines the overall IP address space for the VNet using CIDR notation.
- `--subnet-prefixes`: Defines the IP block for the initial subnet. This must fit within the defined address space.

2. `az network private-endpoint create`

Creates a Private Endpoint, linking a PaaS resource to a private IP in your VNet.

```
az network private-endpoint create \
  --resource-group rg-enterprise-core-prod-001 \
  --name pe-keyvault-prod-001 \
  --vnet-name vnet-spoke-prod-001 \
  --subnet PrivateEndpointSubnet \
  --private-connection-resource-id
"/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg-enterprise-core-prod-0
01/providers/Microsoft.KeyVault/vaults/kv-prod-001" \
  --group-id vault \
  --connection-name conn-kv-prod-001
```

- `--private-connection-resource-id`: The target resource's resource ID. This tells Azure which specific service instance to link.
- `--group-id`: The target sub-resource type (e.g., `vault` for Key Vault, `blob` for Blob Storage, `sqlServer` for SQL).
- `--connection-name`: A descriptive name for the private connection.

3. `az identity create`

Creates a User-Assigned Managed Identity in Entra ID.

```
az identity create \
  --resource-group rg-enterprise-core-prod-001 \
  --name id-spoke-app-prod-001 \
  --location eastus2
```

- `--name`: Sets the name of the identity. This creates a corresponding enterprise application/service principal in Entra ID.

4. `az role assignment create`

Grants access to resources using Role-Based Access Control (RBAC).

```
az role assignment create \
  --assignee 00000000-0000-0000-0000-000000000000 \
  --role "Storage Blob Data Contributor" \
  --scope
"/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg-enterprise-core-prod-0
01"
```

- `--assignee`: The Object ID of the User, Group, Service Principal, or Managed Identity.
- `--role`: The name or ID of the RBAC role to assign.

*

`--scope`: The resource path where the role assignment applies. This can target an entire subscription, a resource group, or a single resource.

---

8. Production Configuration Examples (Terraform HCL)

Below is a production-grade, security-hardened Terraform configuration that implements Hub-Spoke peering, a private endpoint for Key Vault with DNS integration, and strict Network Security Groups (NSGs).

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.90.0"
    }
  }
}

provider "azurerm" {
  features {
    key_vault {
      purge_soft_delete_on_destroy = false
    }
  }
}

variable "location" {
  type    = string
  default = "eastus2"
}

# Resource Group
resource "azurerm_resource_group" "main" {
  name     = "rg-enterprise-core-prod-001"
  location = var.location
  tags = {
    Environment = "Production"
    ManagedBy   = "Terraform"
  }
}

# Hub Network
resource "azurerm_virtual_network" "hub" {
  name                = "vnet-hub-prod-001"
  address_space       = ["10.100.0.0/16"]
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
}
```

```
# Spoke Network
resource "azurerm_virtual_network" "spoke" {
  name                = "vnet-spoke-prod-001"
  address_space       = ["10.101.0.0/16"]
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
}

# Spoke Subnets
resource "azurerm_subnet" "web" {
  name                 = "WebSubnet"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.spoke.name
  address_prefixes     = ["10.101.1.0/24"]
}

resource "azurerm_subnet" "pe" {
  name                 = "PrivateEndpointSubnet"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.spoke.name
  address_prefixes     = ["10.101.2.0/24"]
}

# Global VNet Peering: Hub to Spoke
resource "azurerm_virtual_network_peering" "hub_to_spoke" {
  name                         = "peering-hub-to-spoke"
  resource_group_name          = azurerm_resource_group.main.name
  virtual_network_name         = azurerm_virtual_network.hub.name
  remote_virtual_network_id    = azurerm_virtual_network.spoke.id
  allow_virtual_network_access = true
  allow_forwarded_traffic      = false
  allow_gateway_transit        = true
}

# Global VNet Peering: Spoke to Hub
resource "azurerm_virtual_network_peering" "spoke_to_hub" {
  name                         = "peering-spoke-to-hub"
  resource_group_name          = azurerm_resource_group.main.name
  virtual_network_name         = azurerm_virtual_network.spoke.name
  remote_virtual_network_id    = azurerm_virtual_network.hub.id
  allow_virtual_network_access = true
  allow_forwarded_traffic      = false
  use_remote_gateways          = false
  depends_on                   = [azurerm_virtual_network_peering.hub_to_spoke]
}

# Key Vault
```

```
resource "azurerm_key_vault" "kv" {
  name                        = "kv-prod-ent-001"
  location                    = azurerm_resource_group.main.location
  resource_group_name         = azurerm_resource_group.main.name
  tenant_id                   = "00000000-0000-0000-0000-000000000000" # Replace with real
tenant ID
  sku_name                    = "premium"
  soft_delete_retention_days  = 90
  purge_protection_enabled    = true
  public_network_access_enabled = false

  network_acls {
    bypass         = "AzureServices"
    default_action = "Deny"
  }
}

# Private DNS Zone for Key Vault
resource "azurerm_private_dns_zone" "kv_dns" {
  name                = "privatelink.vaultcore.azure.net"
  resource_group_name = azurerm_resource_group.main.name
}

# Link Private DNS Zone to Spoke VNet
resource "azurerm_private_dns_zone_virtual_network_link" "spoke_dns_link" {
  name                  = "link-spoke-kv-dns"
  resource_group_name   = azurerm_resource_group.main.name
  private_dns_zone_name = azurerm_private_dns_zone.kv_dns.name
  virtual_network_id    = azurerm_virtual_network.spoke.id
}

# Private Endpoint for Key Vault
resource "azurerm_private_endpoint" "kv_pe" {
  name                = "pe-kv-prod-001"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  subnet_id           = azurerm_subnet.pe.id

  private_service_connection {
    name                           = "psc-kv-prod-001"
    private_connection_resource_id = azurerm_key_vault.kv.id
    is_manual_connection           = false
    subresource_names              = ["vault"]
  }

  private_dns_zone_group {
    name                 = "dns-group-kv"
    private_dns_zone_ids = [azurerm_private_dns_zone.kv_dns.id]
```

```
  }
}

# Network Security Group (NSG) with Strict Rules
resource "azurerm_network_security_group" "nsg" {
  name                = "nsg-spoke-web-prod-001"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  # Allow inbound HTTPs within the VNet
  security_rule {
    name                       = "Allow_HTTPS_VNet_Inbound"
    priority                   = 100
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "443"
    source_address_prefix      = "VirtualNetwork"
    destination_address_prefix = "VirtualNetwork"
  }

  # Explicitly deny all other inbound traffic
  security_rule {
    name                       = "Deny_All_Inbound"
    priority                   = 4096
    direction                  = "Inbound"
    access                     = "Deny"
    protocol                   = "*"
    source_port_range          = "*"
    destination_port_range     = "*"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }
}

# Associate NSG with Web Subnet
resource "azurerm_subnet_network_security_group_association" "web_nsg_assoc" {
  subnet_id                 = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.nsg.id
}
```

---

9. Security Considerations & Hardening Best Practices

Identity Hardening (Microsoft Entra ID)

- **Enforce Managed Identities:** Never store client secrets or certificates in application code or configuration repositories. Use System-Assigned or User-Assigned Managed Identities for all Azure resources (VMs, App Services, Functions) that need to access other Azure services.
- **Just-In-Time (JIT) and Privileged Identity Management (PIM):** Do not assign permanent privileged roles (e.g., *Owner*, *User Access Administrator*) to user accounts. Use Entra ID PIM to require multi-factor authentication (MFA), business justification, and manager approval for time-bound, eligible role activations.
- **Conditional Access Policies (CAPs):** Implement strict CAPs that require MFA, compliant devices (Intune enrolled), and trusted IP ranges for all administrative logins. Block legacy authentication protocols entirely.

Network Hardening

- **Zero Public IP Exposure:** Ensure that no VMs or application nodes have public IP addresses. Use Azure Bastion for secure administrative access. Route all outbound internet traffic through a central Azure Firewall Premium instance.
- **Micro-segmentation:** Use Network Security Groups (NSGs) and Application Security Groups (ASGs) to enforce least-privilege traffic flow between subnets. Create rules that explicitly allow traffic between specific components (e.g., Web Subnet to DB Subnet on port 5432) and deny everything else.
- **Private Link Only:** Disable public endpoint access on all PaaS services (Key Vault, Storage Accounts, databases). Use Private Endpoints to ensure all data transfer occurs within your private virtual network space.

Key Vault Hardening

- **Enable Soft-Delete and Purge Protection:** Always enable `soft-delete` and `purge protection` on Key Vaults to prevent accidental or malicious deletion of critical secrets, keys, and certificates.
- **Use Key Vault Access Policies or Azure RBAC:** Prefer Azure RBAC over traditional Key Vault access policies. This allows you to manage control-plane and data-plane access consistently using Entra ID.

---

10. Observability & Monitoring

Metrics to Monitor (Azure Monitor & Prometheus)

| Metric | Target Source | Alert Threshold | Description |

| :--- | :--- | :--- | :--- |

| SNAT Port Exhaustion | Azure Firewall | `> 80%` utilization | High utilization indicates outbound connection limits are being reached, which can lead to dropped connections. |

| Throughput / Bandwidth | ExpressRoute / VPN GW | `> 85%` capacity | Indicates the hybrid network connection is reaching its physical bandwidth limit. |

| CPU / Memory Usage | VMSS Instances | `> 80%` average | Triggers scale-out events to handle increased application load. |

| NSG Flow Drops | Network Security Group | `> 100` drops/min | A sudden spike in dropped packets can indicate a misconfiguration or a potential security scanning attempt. |

Diagnostic Settings & Log Aggregation

All foundational resources must stream diagnostic logs to a central Log Analytics Workspace.

- **Azure Firewall Logs:** Stream `AzureFirewallNetworkRule` and `AzureFirewallApplicationRule` logs to audit allowed and denied connections.
- **NSG Flow Logs:** Enable NSG Flow Logs (Version 2) and aggregate them in a Storage Account. Use Traffic Analytics to analyze IP traffic patterns and identify routing anomalies.

KQL (Kusto Query Language) Troubleshooting Queries

1. Find Blocked Connections in Azure Firewall

```
AzureDiagnostics
| where Category == "AzureFirewallNetworkRule" or Category == "AzureFirewallApplicationRule"
| where Action == "Deny"
| project TimeGenerated, SourceIP, DestinationIP, DestinationPort, Protocol, Action,
RuleCollection = msg_s
| order by TimeGenerated desc
| limit 100
```

2. Analyze NSG Flow Log Packet Drops

```
AzureNetworkAnalyticsIPDetails
| where FlowType == "Inbound" and FlowStatus == "Denied"
| summarize DropCount = count() by SourceIP, DestinationIP, DestinationPort, NSGList_s
| order by DropCount desc
```

---

11. Common Troubleshooting Scenarios

Scenario 1: Private Link DNS Resolution Failure

- **Symptom:** A VM inside the Spoke VNet attempts to access `kv-prod-ent-001.vault.azure.net` but receives its public IP address (which is blocked by the firewall) instead of its private IP (`10.101.2.x`).
- **Root Cause Analysis (RCA):**

1. The Private DNS Zone (`privatelink.vaultcore.azure.net`) is not linked to the Spoke VNet.

2. The VM is configured to use a custom DNS server that does not forward queries to the Azure default DNS IP (`168.63.129.16`).

- **Resolution Steps:**

1. Verify the VNet link exists in the Azure Portal or via CLI:

```
az network private-dns link vnet list --resource-group rg-enterprise-core-prod-001 --zone-name privatelink.vaultcore.azure.net
```

2. If missing, link the Private DNS Zone to the Spoke VNet.

3. If using a custom DNS server (e.g., Active Directory Domain Services), configure conditional forwarders for `vaultcore.azure.net` to forward queries to `168.63.129.16`.

Scenario 2: Asymmetric Routing / Packet Drop with Azure Firewall

- **Symptom:** Workloads in Spoke A cannot establish TCP connections with resources in Spoke B, even though the Azure Firewall rules are configured to allow the traffic.
- **Root Cause Analysis (RCA):**
- Spoke A has a UDR sending traffic (`0.0.0.0/0`) to the Azure Firewall.
- However, Spoke A and Spoke B are directly peered.
- Because of this direct peering, the return path from Spoke B to Spoke A bypasses the firewall, routing directly over the peering link.
- The firewall drops the initial connection because it sees the request but never receives the return traffic (violating TCP state alignment).
- **Resolution Steps:**
- Add a specific route to the Route Table in Spoke A for Spoke B's IP range (e.g., `10.102.0.0/16`) with a next hop of `VirtualAppliance` pointing to the Azure Firewall IP.
- Alternatively, remove the direct peering between Spoke A and Spoke B, forcing all transit traffic through the Hub Firewall.

Scenario 3: ARM API Throttling (HTTP 429 Too Many Requests)

- **Symptom:** During a major traffic spike, VMSS fails to scale out, and the activity logs show `HTTP 429` errors from the ARM control plane.
- **Root Cause Analysis (RCA):**
- An automated script, monitoring agent, or rapid scaling event is making too many concurrent read/write calls to the ARM API, exceeding the per-subscription rate limits (typically 12,000 read requests and 1,200 write requests per hour).
- **Resolution Steps:**
- Implement exponential backoff and retry logic in all automation scripts and CI/CD pipelines.
- Consolidate API calls using bulk operations.
- Distribute resources across multiple subscriptions if the workload naturally hits these limits due to scale.

---

12. Common Mistakes and How to Avoid Them

- **Using Service Endpoints instead of Private Endpoints for Multi-Tenant PaaS:**
- *Mistake:* Configuring Service Endpoints on a subnet, assuming it is secure.
- *Risk:* A compromised VM can write data to an external, attacker-controlled storage account of the same type, bypassing security controls.
- *Correction:* Always use Private Endpoints for production environments. This completely disables public network access and restricts communication to the specific resource instance.
- **Overlapping CIDR Blocks:**
- *Mistake:* Allocating overlapping IP ranges (e.g., `10.100.0.0/16` for both Hub and Spoke VNets).
- *Risk:* You cannot peer VNets with overlapping IP addresses. Resolving this later requires recreating the VNets and redeploying all associated resources.
- *Correction:* Maintain a central IP Address Management (IPAM) registry. Design a clear, non-overlapping subnet allocation strategy before deploying any network infrastructure.
- **Hardcoding Credentials in App Settings or Pipelines:**
- *Mistake:* Storing Service Principal client secrets in application configurations, App Service settings, or pipeline variables.
- *Risk:* Credentials can be leaked in application logs, source control, or to unauthorized users.
- *Correction:* Use Managed Identities for Azure resources. For external pipelines, use OpenID Connect (OIDC) federated credentials to authenticate with Azure without using secrets.

---

13. Enterprise-Level Recommendations

Performance Tuning: Accelerated Networking (SR-IOV)

- Always enable Accelerated Networking on supported VM sizes.
- This feature bypasses the virtual switch, reducing latency, jitter, and CPU utilization on the host. It is critical for high-throughput workloads like databases, Kubernetes nodes, and network virtual appliances.

```
[Standard Networking]
VM ??? Virtual Switch (Host CPU Overhead) ??? Physical NIC

[Accelerated Networking (SR-IOV)]
VM ??? Physical NIC (Direct Hardware Access, Bypass Switch)
```

DNS Forwarding Topologies (Azure DNS Private Resolver)

- In hybrid cloud environments, resolving on-premises domains from Azure (and vice versa) can be challenging.
- Avoid deploying and maintaining pairs of DNS forwarder VMs. Instead, use the **Azure DNS Private Resolver** service. This cloud-native, highly available service handles DNS forwarding rules across hybrid connections without the management overhead of virtual machines.

ARM Deployment Optimization

- Use Azure Bicep or Terraform to modularize your infrastructure deployments.
- Group resources by lifecycle rather than type. For example, place your core networking infrastructure in one state file/repository, and deploy application-specific resources (VMSS, Private Endpoints) in separate, decoupled pipelines.

---

14. Advanced Concepts

ARM Engine Deep Dive

The Azure Resource Manager (ARM) is the control plane for Azure. When you submit a deployment (via Portal, CLI, Terraform, or Bicep):

1. Authentication & Authorization: The request is authenticated via Entra ID, and RBAC policies are checked to ensure you have permissions at the target scope.

2. Routing to Resource Providers: ARM parses the request and routes it to the appropriate Resource Provider (RP) (e.g., `Microsoft.Network` for VNets, `Microsoft.Compute` for VMs).

3. Idempotency & State Machine: RPs process the declarative JSON template. If a resource already exists and matches the configuration, no action is taken. If there are differences, the RP updates the resource to match the desired state.

BGP Propagation Mechanics in ExpressRoute

- When connecting on-premises datacenters to Azure via ExpressRoute, routing tables are exchanged dynamically using the Border Gateway Protocol (BGP).
- **Gateway Subnet UDRs:** By default, Azure routes traffic to on-premises networks using the BGP routes learned by the virtual network gateway. If you need to inspect traffic going to on-premises, you can apply a UDR to the GatewaySubnet. This forces traffic destined for your local network to route through an intermediate appliance (like Azure Firewall) first.

---

15. Integration with Other DevOps Tools

CI/CD Pipelines (GitHub Actions & OIDC Federated Credentials)

Avoid storing static Azure Service Principal secrets in GitHub. Instead, use OpenID Connect (OIDC) to authenticate your workflows.

```
# .github/workflows/deploy-infra.yml
name: Deploy Azure Infrastructure
on:
  push:
    branches: [ main ]

permissions:
  id-token: write # Required for requesting the JWT
  contents: read  # Required for actions/checkout

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Azure Login via OIDC
        uses: azure/login@v1
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - name: Terraform Init & Apply
        run: |
          terraform init
          terraform apply -auto-approve
```

Ansible Dynamic Inventory

Use the Azure dynamic inventory plugin (`azure_rm`) to automatically discover VMs and VMSS instances in your subscription based on tags, rather than maintaining static IP lists.

```
# inventory.azure_rm.yml
plugin: azure_rm
auth_source: auto
include_vmss_resource_groups:
  - rg-enterprise-core-prod-001
keyed_groups:
  - key: tags.Role
    prefix: role
```

---

16. Multi-Cloud Comparison

| Service Category | Azure | AWS | GCP | Architectural Differences & Trade-offs |

| :--- | :--- | :--- | :--- | :--- |

| Virtual Network | Virtual Network (VNet) | Virtual Private Cloud (VPC) | VPC Network | * Azure: VNets are regional; you use peering to connect them globally.
* GCP: VPCs are global resources by default.
* AWS: VPCs are regional, and subnets are tied to specific Availability Zones. |

| Private Connectivity | Private Link / Private Endpoint | VPC Endpoint (Interface / Gateway) | Private Service Connect | * Azure/AWS: Allocate a private IP inside the network to access PaaS services.
* GCP: Uses global internal load balancing to route traffic privately. |

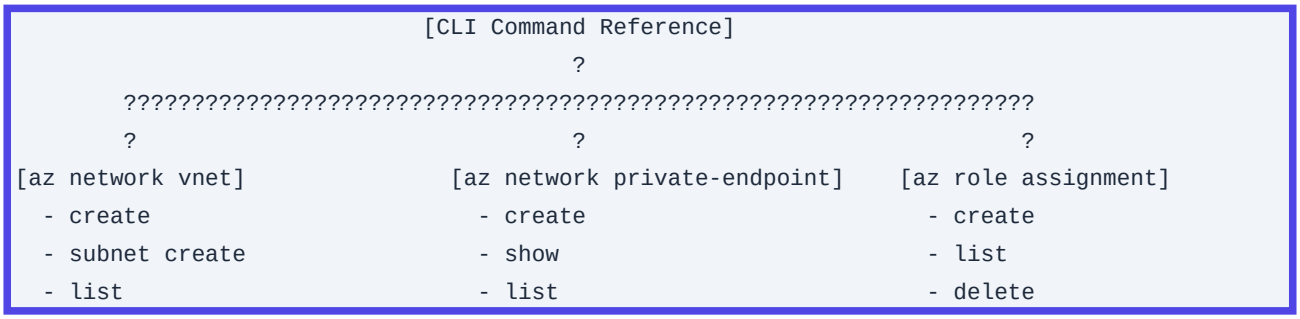
| Identity Engine | Microsoft Entra ID | AWS IAM | Google Cloud IAM | * Azure: Centrally managed at the tenant level, natively integrating with enterprise directories.
* AWS/GCP: Account/Project-centric models. They require external federation or configuration to manage identities across multiple accounts. |

| Firewall | Azure Firewall Premium | AWS Network Firewall | Cloud Firewall | * Azure: Highly integrated with ARM, offering native TLS inspection and IDPS capabilities.
* AWS/GCP: Offer similar features, but often require third-party integrations (e.g., Palo Alto, Fortinet) for advanced deep packet inspection. |

---

17. Visual Cheat Sheet

Core CLI Commands



- `az network vnet create -g <rg> -n <name> --address-prefixes <CIDR>`
- `az network private-endpoint create -g <rg> -n <name> --vnet-name <vnet> --subnet <subnet> --private-connection-resource-id <id> --group-id <sub-resource>`
- `az role assignment create --assignee <object-id> --role <role-name> --scope <scope-id>`

Subnet Allocation Rules

- **AzureFirewallSubnet:** Minimum `/26` required. Must be named exactly `AzureFirewallSubnet`.
- **GatewaySubnet:** Minimum `/27` recommended. Must be named exactly `GatewaySubnet`. Do not associate NSGs or Route Tables with this subnet.
- **AzureBastionSubnet:** Minimum `/26` required. Must be named exactly `AzureBastionSubnet`.

Port & Protocol Rules

- **Azure Bastion Inbound:** Port `443` from the internet.
- **Azure Bastion Outbound:** Port `443` and `22/3389` to the target VM subnets.
- **Private Endpoint Inbound:** Port associated with the target PaaS service (e.g., `443` for Key Vault/Blob, `1433` for Azure SQL).

---

18. Comprehensive Final Learning Summary

6-Month Progression Roadmap

```
Month 1: Core Foundations (Networking, Identity, ARM Engine)  ??? [CURRENTLY HERE]
  ?
  ??? Month 2-3: Advanced Compute & Storage (AKS, App Services, Blob, Cosmos DB)
  ?
  ??? Month 4-5: Enterprise Security & Governance (Azure Policy, Defender, Sentinel)
  ?
  ??? Month 6: Advanced DevOps Automation (Complex CI/CD, Custom IaC, Site Reliability)
```

Key Takeaways

1. Default Deny: Design your networks with a zero-trust mindset. Disable public endpoints, use Private Link, and route all outbound traffic through Azure Firewall Premium.

2. Identity is the Perimeter: Move away from shared keys and passwords. Use Managed Identities and enforce strict RBAC policies with PIM.

3. Design for Failure: Avoid single points of failure. Deploy workloads across multiple Availability Zones, use zone-redundant network gateways, and automate your deployments using Infrastructure as Code (IaC) to ensure rapid, consistent recovery.

Q1. Explain the Azure Resource Manager (ARM) architecture. How does ARM handle request throttling, and how can an SRE programmatically register resource providers and inspect API limits?

Detailed Answer:

Azure Resource Manager (ARM) is the deployment and management service for Azure. It provides a consistent management layer (via REST APIs) that enables you to create, update, and delete resources in your Azure subscription. When a user sends a request from any Azure tool (Portal, CLI, PowerShell, SDKs, or IaC tools like Terraform/Bicep), ARM intercepts the request, authenticates and authorizes it via Microsoft Entra ID (formerly Azure AD), and routes it to the appropriate Azure Resource Providers (RPs) (e.g., `Microsoft.Compute`, `Microsoft.Network`).

ARM enforces rate limiting (throttling) to ensure high availability and multi-tenant isolation. Subscription-level read requests are limited to 12,000 per hour, and write requests are limited to 1,200 per hour. Tenant-level limits also apply. When these limits are exceeded, ARM returns HTTP Status Code `429 Too Many Requests` along with a `Retry-After` header indicating the number of seconds the client must wait before retrying.

To prevent deployment failures, an SRE must ensure that all required Resource Providers are registered within the target subscription before deploying IaC templates. This can be handled programmatically. To handle throttling, clients should implement exponential backoff retry policies and monitor the `x-ms-ratelimit-remaining-subscription-reads` and `x-ms-ratelimit-remaining-subscription-writes` response headers.

Production Scenario / Practical Example:

An enterprise bootstrap script written in Azure CLI registers required resource providers for a new subscription and inspects the remaining API rate limits.

```
# Register required resource providers for a standard enterprise landing zone
az provider register --namespace Microsoft.Compute --wait
az provider register --namespace Microsoft.Network --wait
az provider register --namespace Microsoft.Storage --wait
az provider register --namespace Microsoft.ContainerService --wait

# Show registration status of a specific provider
az provider show --namespace Microsoft.Compute --query "registrationState" -o tsv

# Query current subscription-level API rate limits by inspecting response headers
# We execute a dummy call with --debug to extract the rate-limit headers
az group list --debug 2>&1 | grep -i "x-ms-ratelimit-remaining-subscription"
```

Q2. Describe the Azure Landing Zone (ALZ) Management Group Hierarchy. How does resource inheritance work, and how do you programmatically move subscriptions between Management Groups?

Detailed Answer:

An Azure Landing Zone (ALZ) is an architectural framework based on Microsoft's Cloud Adoption Framework (CAF). It relies on a hierarchical structure of Management Groups (MGs) to enforce governance, security policies, and compliance at scale. The hierarchy typically starts with the *Tenant Root Group*, under which an

Intermediate Management Group (representing the organization) is created. Below this, the hierarchy splits into operational archetypes:

- **Platform**: Contains sub-MGs like *Identity* (Domain Controllers), *Management* (Log Analytics, Sentinel), and *Connectivity* (Hub VNets, ExpressRoute).
- **Workloads**: Contains sub-MGs like *Production* and *Non-Production* to isolate application environments.
- **Sandbox**: For R&D, where policies are relaxed.

Inheritance flows downwards. Any Azure Policy, RBAC role assignment, or budget applied at a higher MG is automatically inherited by child MGs and subscriptions. You cannot block inheritance; you can only append policies or roles.

Moving a subscription between MGs requires write permission at both the source and destination MGs (specifically, the permission `Microsoft.Management/managementGroups/write` or `Microsoft.Authorization/roleAssignments/write`).

Production Scenario / Practical Example:

The following script moves an active subscription from the `Sandbox` Management Group to the `Workloads-Prod` Management Group using the Azure CLI.

```
# Define Management Group IDs and Subscription ID
TENANT_ID="11111111-2222-3333-4444-555555555555"
DEST_MG_ID="org-workloads-prod"
SUB_ID="66666666-7777-8888-9999-000000000000"

# Verify destination management group exists
az account management-group show --name "$DEST_MG_ID"

# Associate the subscription to the destination Management Group
# This automatically removes it from its current parent group
az account management-group subscription add \
  --name "$DEST_MG_ID" \
  --subscription "$SUB_ID"

# Verify the subscription placement
az account management-group subscription show \
  --name "$DEST_MG_ID" \
  --subscription "$SUB_ID"
```

Q3. Explain Azure Virtual Network (VNet) Peering. What are the key configuration options (`AllowVirtualNetworkAccess`, `AllowForwardedTraffic`, `AllowGatewayTransit`, `UseRemoteGateways`), and how do you resolve CIDR conflicts?

Detailed Answer:

VNet Peering directly connects two Azure Virtual Networks using the Microsoft backbone network, enabling resources in either VNet to communicate with low latency and high bandwidth. VNet Peering is non-transitive; if VNet A is peered with VNet B, and VNet B is peered with VNet C, VNet A cannot communicate with VNet C unless a direct peering is established or a Network Virtual Appliance (NVA)/VPN Gateway routes the traffic.

Key parameters when establishing a peering include:

1. AllowVirtualNetworkAccess: Enables VM-to-VM communication between the peered VNets. Must be set to `true` on both sides for bidirectional communication.

2. AllowForwardedTraffic: Allows traffic originating from outside the peered VNet (e.g., forwarded by an NVA or loopback) to enter the VNet. Crucial for hub-and-spoke topologies with central firewalls.

3. AllowGatewayTransit: Configured on the Hub VNet (which contains the Virtual Network Gateway). It allows the Hub's gateway to share its VPN/ExpressRoute connection with the spoke VNets.

4. UseRemoteGateways: Configured on the Spoke VNet. It allows the Spoke to leverage the Hub VNet's gateway to access on-premises networks.

CIDR Conflicts: VNets with overlapping IP address spaces cannot be peered. If an overlap occurs, you must either recreate the VNet with a non-overlapping CIDR, or use Azure Virtual Network Manager (AVNM) / NAT rules on an intermediate NVA/Gateway to translate overlapping addresses.

Production Scenario / Practical Example:

Below is a Bicep configuration establishing a peer relationship from a Spoke VNet to a Hub VNet, enabling remote gateway usage and forwarded traffic.

```
// Spoke to Hub Peering Configuration (deployed in Spoke VNet scope)
resource spokeToHubPeering
'Microsoft.Network/virtualNetworks/virtualNetworkPeerings@2023-09-01' = {
  name: 'vnet-spoke-01/peering-to-hub'
  properties: {
    allowVirtualNetworkAccess: true
    allowForwardedTraffic: true
    allowGatewayTransit: false
    useRemoteGateways: true // Leverages the Hub VNet's gateway
    remoteVirtualNetwork: {
      id: resourceId('rg-connectivity-hub', 'Microsoft.Network/virtualNetworks',
'vnet-hub-prod')
    }
  }
}
```

Q4. Compare Azure RBAC Roles and Microsoft Entra ID (Azure AD) Roles. How does the

control plane differ from the data plane, and how do you write a custom RBAC JSON definition to enforce least privilege?

Detailed Answer:

Azure RBAC (Role-Based Access Control) and Microsoft Entra ID (Azure AD) roles govern different security boundaries:

- **Azure RBAC Roles**: Manage access to *Azure Resources* (e.g., VMs, VNets, Storage Accounts). These roles are applied at the Management Group, Subscription, Resource Group, or Resource scope.
- **Microsoft Entra ID Roles**: Manage *Tenant-level Identity* configurations (e.g., creating users, registering applications, managing enterprise apps, assigning licenses).

Control Plane vs. Data Plane:

- **Control Plane (Management Plane)**: Actions that modify the resource metadata or configuration. Example: `Microsoft.Storage/storageAccounts/write` (creating a storage account).
- **Data Plane**: Actions that interact with the data stored inside the resource. Example: `Microsoft.Storage/storageAccounts/blobServices/containers/blobs/read` (reading a file inside a blob container). Traditional Azure RBAC roles (like Owner/Contributor) only govern the control plane unless they explicitly include data plane actions (like Storage Blob Data Contributor).

To enforce the principle of least privilege, SREs define custom RBAC roles using JSON, specifying precise actions, data actions, and assignable scopes.

Production Scenario / Practical Example:

The following JSON defines a custom Azure RBAC role named "SRE Network Operator". It allows reading network configurations and restarting VMs, but prevents modifying NSGs or deleting resources.

```
{
  "Name": "SRE Network Operator",
  "IsCustom": true,
  "Description": "Allows read-only network access and VM restarts, but blocks NSG
mutations.",
  "Actions": [
    "Microsoft.Network/*/read",
    "Microsoft.Compute/virtualMachines/read",
    "Microsoft.Compute/virtualMachines/restart/action"
  ],
  "NotActions": [
    "Microsoft.Network/networkSecurityGroups/*"
  ],
  "DataActions": [],
  "NotDataActions": [],
  "AssignableScopes": [
    "/subscriptions/66666666-7777-8888-9999-000000000000"
  ]
}
```

To apply this custom role via Azure CLI:

```
az role definition create --role-definition ./sre-network-operator.json
```

Q5. Explain the mechanics of Azure Policy. What are the key policy effects (`Deny`, `AuditIfNotExists`, `DeployIfNotExists`), and how do you write a policy to enforce that all Subnets have a Network Security Group (NSG) associated?

Detailed Answer:

Azure Policy evaluates resources at various points in their lifecycle (creation, update, and during compliance evaluation cycles).

- **Deny**: Evaluated before the resource reaches the Resource Provider. If the resource configuration violates the policy, ARM blocks the deployment and returns a `RequestDisallowedByPolicy` error.
- **AuditIfNotExists**: Evaluated after the resource provider has processed the request. It checks if a related sub-resource or child resource exists (e.g., checking if diagnostic settings exist on a VM). If it does not, the resource is flagged as non-compliant, but deployment is not blocked.
- **DeployIfNotExists (DINE)**: Similar to `AuditIfNotExists`, but if the target sub-resource is missing, Azure Policy deploys a configured ARM template to remediate the resource automatically. This requires a Managed Identity with sufficient RBAC permissions.

To enforce security compliance, policies are grouped into Initiatives (Policy Sets) and assigned at the subscription or management group level.

Production Scenario / Practical Example:

An enterprise Azure Policy definition (JSON) that denies the creation of subnets that do not have a Network Security Group associated.

```
{
  "properties": {
    "displayName": "Enforce Network Security Group association on Subnets",
    "policyType": "Custom",
    "mode": "All",
    "description": "This policy denies the creation or update of any subnet that does not
have an associated Network Security Group.",
    "metadata": {
      "category": "Network"
    },
    "policyRule": {
      "if": {
        "allOf": [
          {
            "field": "type",
            "equals": "Microsoft.Network/virtualNetworks/subnets"
```

```
          },
          {
            "anyOf": [
              {
                "field": "Microsoft.Network/virtualNetworks/subnets/networkSecurityGroup.id",
                "exists": "false"
              },
              {
                "field": "Microsoft.Network/virtualNetworks/subnets/networkSecurityGroup.id",
                "equals": ""
              }
            ]
          }
        ]
      },
      "then": {
        "effect": "deny"
      }
    }
  }
}
```

Q6. Deep dive into Azure Private Link and Private Endpoints. How do they differ from Service Endpoints, how does DNS integration work, and how do you avoid split-brain DNS issues?

Detailed Answer:

- **Service Endpoints**: Extend your VNet's private address space and identity to Azure PaaS services over the Microsoft backbone. The PaaS service still retains its public IP address; the traffic is simply routed optimally. You cannot access the service from on-premises networks via ExpressRoute/VPN using Service Endpoints.
- **Private Endpoints**: Inject a physical network interface (NIC) with a private IP address from your VNet subnet directly into the VNet. Traffic to the PaaS service (e.g., Azure SQL, Storage) is routed directly to this private IP. This allows secure, private access from both within the VNet and from on-premises networks over ExpressRoute/VPN.

DNS Integration & Avoiding Split-Brain DNS:

Because PaaS services use public DNS names (e.g., `mystorage.blob.core.windows.net`), clients must resolve this FQDN to the Private Endpoint's private IP (`10.0.1.5`) when inside the VNet, but public clients should still resolve it to the public IP (if public access is enabled).

To achieve this without split-brain DNS issues:

1. Azure uses a canonical name (CNAME) record pointing to a private link zone (e.g., `mystorage.privatelink.blob.core.windows.net`).

2. You must provision an Azure Private DNS Zone named `privatelink.blob.core.windows.net` and link it to your VNets.

3. An `A` record is created in this Private DNS Zone mapping `mystorage` to the private IP.

4. On-premises DNS servers must configure conditional forwarders pointing to an Azure DNS Private Resolver to resolve these private IPs.

Production Scenario / Practical Example:

Here is an ARM/Bicep snippet illustrating the creation of a Private Endpoint for an Azure Blob Storage account, linking it to a Private DNS Zone.

```
// Private Endpoint for Storage Account Blob Service
resource privateEndpoint 'Microsoft.Network/privateEndpoints@2023-09-01' = {
  name: 'pe-storage-blob'
  location: resourceGroup().location
  properties: {
    subnet: {
      id:
'/subscriptions/66666666-7777-8888-9999-000000000000/resourceGroups/rg-network/providers/Micr
osoft.Network/virtualNetworks/vnet-spoke/subnets/sn-endpoints'
    }
    privateLinkServiceConnections: [
      {
        name: 'conn-storage-blob'
        properties: {
          privateLinkServiceId: resourceId('Microsoft.Storage/storageAccounts',
'sa-prod-data')
          groupIds: [
            'blob'
          ]
        }
      }
    ]
  }
}

// Associate Private Endpoint with Private DNS Zone
resource dnsZoneGroup 'Microsoft.Network/privateEndpoints/privateDnsZoneGroups@2023-09-01' =
{
  parent: privateEndpoint
  name: 'default'
  properties: {
    privateDnsZoneConfigs: [
      {
        name: 'privatelink-blob-core-windows-net'
        properties: {
          privateDnsZoneId: resourceId('rg-dns', 'Microsoft.Network/privateDnsZones',
'privatelink.blob.core.windows.net')
```

```
        }
      }
    ]
  }
}
```

Q7. What is Azure NAT Gateway? How does it prevent SNAT Port Exhaustion compared to default outbound rules or load balancer outbound configurations?

Detailed Answer:

When VMs in a private subnet initiate outbound connections to the internet (e.g., for software updates or API calls), they require Source Network Address Translation (SNAT).

- **Default Outbound Access / Load Balancer SNAT**: Azure assigns a dynamic, pre-allocated block of SNAT ports (typically 1,024 ports per VM) from the load balancer's public IP pool. If a VM opens hundreds of concurrent connections (e.g., high-throughput microservices, database connections, or connection pools that do not close properly), it quickly exhausts these 1,024 ports, leading to connection drops (SNAT Port Exhaustion).
- **Azure NAT Gateway**: NAT Gateway is a fully managed, highly resilient service assigned directly to a subnet. It provides a shared pool of up to 64,000 SNAT ports per public IP address (up to 16 public IPs can be associated, yielding ~1,024,000 dynamic ports). Instead of static allocation per VM, NAT Gateway dynamically allocates SNAT ports from the entire pool to any VM in the subnet on demand. This virtually eliminates SNAT port exhaustion.

Production Scenario / Practical Example:

This Azure CLI snippet provisions a NAT Gateway, associates it with a Static Public IP, and attaches it to an existing Subnet.

```
# Create a Standard Public IP for the NAT Gateway
az network public-ip create \
  --resource-group rg-network-prod \
  --name pip-natgw-prod \
  --sku Standard \
  --zone 1 2 3 \
  --allocation-method Static

# Create the NAT Gateway
az network nat gateway create \
  --resource-group rg-network-prod \
  --name natgw-prod \
  --public-ip-addresses pip-natgw-prod \
  --idle-timeout 4

# Associate the NAT Gateway with an existing Subnet
```

```
az network vnet subnet update \
  --resource-group rg-network-prod \
  --vnet-name vnet-spoke-prod \
  --name sn-app-tier \
  --nat-gateway natgw-prod
```

Q8. Explain Azure Storage Account security. How do you implement Storage SAS Tokens safely, configure Storage Firewalls, and establish object replication and lifecycle management policies?

Detailed Answer:

Azure Storage Accounts must be secured using defense-in-depth principles:

1. Network Isolation: Disable public access. Configure the Storage Firewall to allow access only from selected VNets (via Service Endpoints/Private Endpoints) and trusted IP ranges.

2. Authentication: Disable Shared Key authentication in favor of Microsoft Entra ID (RBAC). If Shared Access Signatures (SAS) must be used, generate User Delegation SAS tokens (secured by Entra ID credentials) rather than Service/Account SAS tokens (which rely on the master account key).

3. Data Protection & Lifecycle: Use Lifecycle Management to automate tiering (Hot to Cool/Cold/Archive) and purge expired objects. Use Object Replication for block blobs to asynchronously replicate data to another storage account across regions for disaster recovery.

Production Scenario / Practical Example:

The following Bicep code configures a secure Storage Account that blocks public access except from a specific subnet, disables Shared Key access, and configures a lifecycle management policy to transition blobs to Archive after 90 days.

```
resource storageAccount 'Microsoft.Storage/storageAccounts@2023-01-01' = {
  name: 'saproddataaudit'
  location: resourceGroup().location
  sku: {
    name: 'Standard_GRS'
  }
  kind: 'StorageV2'
  properties: {
    allowBlobPublicAccess: false
    minimumTlsVersion: 'TLS1_2'
    supportsHttpsTrafficOnly: true
    allowSharedKeyAccess: false // Enforces Microsoft Entra ID auth
    networkAcls: {
      bypass: 'Logging, Metrics'
      defaultAction: 'Deny' // Restricts public access
      virtualNetworkRules: [
```

```
        {
          id:
'/subscriptions/66666666-7777-8888-9999-000000000000/resourceGroups/rg-network/providers/Micr
osoft.Network/virtualNetworks/vnet-spoke/subnets/sn-app-tier'
          action: 'Allow'
        }
      ]
    }
  }
}

resource lifecyclePolicy 'Microsoft.Storage/storageAccounts/managementPolicies@2023-01-01' =
{
  parent: storageAccount
  name: 'default'
  properties: {
    policy: {
      rules: [
        {
          enabled: true
          name: 'ArchiveOldLogs'
          type: 'Lifecycle'
          definition: {
            actions: {
              baseBlob: {
                tierToCool: { daysAfterModificationGreaterThan: 30 }
                tierToArchive: { daysAfterModificationGreaterThan: 90 }
                delete: { daysAfterModificationGreaterThan: 365 }
              }
            }
            filters: {
              blobTypes: [ 'blockBlob' ]
              prefixMatch: [ 'logs/' ]
            }
          }
        }
      ]
    }
  }
}
```

Q9. Compare Azure Virtual Machine Scale Sets (VMSS) Uniform vs. Flexible Orchestration Modes. How do you implement spot instance eviction handling, and how do you configure manual vs. custom autoscale rules?

Detailed Answer:

Azure VMSS supports two orchestration modes:

- **Uniform Orchestration**: Uses a single, standardized VM template. Scale actions are highly optimized, making it ideal for massive, homogeneous workloads (e.g., stateless web farms, container nodes).
- **Flexible Orchestration**: Allows you to mix VM sizes, families, and spot/on-demand instances within the same scale set. It provides control over individual VMs and supports attaching existing VMs. It offers high availability by distributing VMs across fault domains within a region.

Spot Instance Eviction: Spot VMs utilize spare Azure capacity at deep discounts but can be evicted with a 30-second warning when Azure needs the capacity back. To handle this, SREs subscribe to Scheduled Events via the Instance Metadata Service (IMDS) endpoint (`http://169.254.169.254/metadata/scheduledevents?api-version=2020-07-01`) or configure VMSS eviction policies to either "Deallocate" (saves disk state) or "Delete" (purges the VM completely).

Autoscale Configurations:

- **Manual**: Fixed instance count defined directly on the VMSS.
- **Custom Autoscale**: Dynamic scaling based on metrics (e.g., CPU, Memory, Queue Depth, or custom Azure Monitor metrics) with cool-down periods to prevent thrashing.

Production Scenario / Practical Example:

An autoscale rule configuration (JSON used via CLI) that scales a VMSS out by 2 instances if average CPU exceeds 80% for 5 minutes, and scales in by 1 instance if CPU falls below 30%.

```
{
  "rules": [
    {
      "metricTrigger": {
        "metricName": "Percentage CPU",
        "metricNamespace": "microsoft.compute/virtualmachinescalesets",
        "metricResourceUri":
"/subscriptions/66666666-7777-8888-9999-000000000000/resourceGroups/rg-compute/providers/Micr
osoft.Compute/virtualMachineScaleSets/vmss-app-prod",
        "timeGrain": "PT1M",
        "statistic": "Average",
        "timeWindow": "PT5M",
        "timeAggregation": "Average",
        "operator": "GreaterThan",
        "threshold": 80.0
      },
      "scaleAction": {
        "direction": "Increase",
        "type": "ChangeCount",
        "value": "2",
        "cooldown": "PT5M"
      }
    },
```

```
    {
      "metricTrigger": {
        "metricName": "Percentage CPU",
        "metricNamespace": "microsoft.compute/virtualmachinescalesets",
        "metricResourceUri":
"/subscriptions/66666666-7777-8888-9999-000000000000/resourceGroups/rg-compute/providers/Micr
osoft.Compute/virtualMachineScaleSets/vmss-app-prod",
        "timeGrain": "PT1M",
        "statistic": "Average",
        "timeWindow": "PT5M",
        "timeAggregation": "Average",
        "operator": "LessThan",
        "threshold": 30.0
      },
      "scaleAction": {
        "direction": "Decrease",
        "type": "ChangeCount",
        "value": "1",
        "cooldown": "PT5M"
      }
    }
  ]
}
```

Q10. What is Azure Bastion? How does the architecture work, how do NSG rules apply to the `AzureBastionSubnet`, and how do you initiate a native SSH/RDP session using the Azure CLI?

Detailed Answer:

Azure Bastion is a fully platform-managed PaaS service that provides secure, seamless RDP and SSH access to VMs directly over TLS (port 443) from the Azure Portal or native client tools. It eliminates the need to expose public IP addresses on your target VMs.

Architectural Requirements:

1. Dedicated Subnet: Must be named exactly `AzureBastionSubnet` and be at least `/26` in size.
2. NSG Rules: The NSG associated with the `AzureBastionSubnet` must strictly permit:
   - *Inbound*: Port 443 from `Internet` (for user traffic), Port 443 from `GatewayManager` (for control plane operations), and Ports 5701/8080 from `VirtualNetwork` (for internal clustering).
   - *Outbound*: Ports 3389 (RDP) and 22 (SSH) to `VirtualNetwork`, Port 443 to `AzureCloud` (for control plane endpoints), and Ports 5701/8080 to `VirtualNetwork` (internal mesh).

Using the Azure Bastion Developer or Standard SKU, SREs can tunnel native SSH/RDP connections from their local terminal, avoiding the Azure Portal entirely.

Production Scenario / Practical Example:

The following commands log into Azure, configure a tunnel, and establish a native SSH connection to a private VM using Azure Bastion.

```
# Log in and set target subscription
az account set --subscription "66666666-7777-8888-9999-000000000000"

# Target VM details
VM_RG="rg-compute-prod"
VM_NAME="vm-backend-01"
BASTION_NAME="bas-prod-hub"
BASTION_RG="rg-connectivity-hub"

# Establish native SSH session using Azure Bastion tunnel
az network bastion ssh \
  --name "$BASTION_NAME" \
  --resource-group "$BASTION_RG" \
  --target-resource-id
"/subscriptions/66666666-7777-8888-9999-000000000000/resourceGroups/$VM_RG/providers/Microsof
t.Compute/virtualMachines/$VM_NAME" \
  --auth-type "ssh-key" \
  --username "sreadmin" \
  --ssh-key "~/.ssh/id_rsa"
```

Q11. Contrast Azure Key Vault Access Policies and Azure RBAC permission models. How do you implement network isolation for Key Vault, and how do you retrieve a secret securely within a pipeline?

Detailed Answer:

Azure Key Vault supports two authorization models:

1. Vault Access Policy: A legacy, coarse-grained permission model applied globally at the vault level. It does not support scope-level isolation (e.g., you cannot grant permission to read only a single secret; users get read access to all secrets in the vault).

2. Azure RBAC Model (Recommended): A modern, fine-grained model aligned with standard Azure IAM. It allows assigning roles (like *Key Vault Secrets Reader* or *Key Vault Secrets Officer*) at the Vault level, the secret/key/certificate folder level, or even down to individual secret objects.

Network Isolation: Key Vault should have public access disabled and be accessed via a Private Endpoint. Additionally, you can configure the Key Vault firewall to restrict access to trusted Azure Services (e.g., Azure Pipelines hosted agents or specific VNets).

Production Scenario / Practical Example:

The following Azure CLI script creates a Key Vault using the Azure RBAC permission model, restricts access to a private VNet, and retrieves a secret using a Managed Identity.

```
# Create a secure Key Vault using Azure RBAC authorization
az keyvault create \
  --name "kv-prod-secrets-01" \
  --resource-group "rg-sec-prod" \
  --location "eastus2" \
  --enable-rbac-authorization true \
  --default-action Deny # Blocks all public traffic

# Allow access from the App Subnet Service Endpoint
az keyvault network-rule add \
  --name "kv-prod-secrets-01" \
  --vnet-name "vnet-spoke-prod" \
  --subnet "sn-app-tier"

# Retrieve secret in a pipeline (assuming pipeline agent runs with a Managed Identity)
# First, authenticate as the Managed Identity
az login --identity

# Retrieve the secret value directly (requires Key Vault Secrets User role)
SECRET_VAL=$(az keyvault secret show \
  --vault-name "kv-prod-secrets-01" \
  --name "db-connection-string" \
  --query "value" -o tsv)
```

Q12. Explain Azure DNS Private Zones. How does auto-registration work, how do you manage DNS resolution across multiple peered VNets, and what is the role of the Azure DNS Private Resolver?

Detailed Answer:

Azure DNS Private Zones provide a reliable, secure DNS service to resolve domain names inside a virtual network without needing custom DNS servers.

- **Virtual Network Links**: To make a Private DNS Zone resolvable by a VNet, you must link the zone to the VNet using a Virtual Network Link.
- **Auto-Registration**: When linking a VNet, you can enable auto-registration (`registrationEnabled: true`). This automatically creates DNS `A` records in the zone whenever a VM is deployed in that VNet, and deletes them when the VM is destroyed. *Constraint*: A VNet can only be linked to one Private DNS Zone for auto-registration at a time, though it can be linked to multiple zones for resolution.
- **Resolution across Peered VNets**: If VNet A and VNet B are peered, resources in VNet B cannot resolve names in VNet A's private zone unless VNet B is also explicitly linked to that same Private DNS Zone.

- **Azure DNS Private Resolver**: A fully managed service that bridges hybrid DNS resolution. It provides **Inbound Endpoints** (allowing on-premises DNS to forward queries to Azure DNS) and **Outbound Endpoints** (allowing Azure VNets to forward queries to on-premises DNS based on DNS Forwarding Rulesets).

Production Scenario / Practical Example:

Here is a Bicep template creating a Private DNS Zone (`corp.internal`), linking it to a Spoke VNet with auto-registration enabled, and linking it to a Hub VNet with resolution-only access.

```
// Define the Private DNS Zone
resource privateDnsZone 'Microsoft.Network/privateDnsZones@2020-06-01' = {
  name: 'corp.internal'
  location: 'global'
}

// Link to Spoke VNet with Auto-Registration Enabled
resource spokeLink 'Microsoft.Network/privateDnsZones/virtualNetworkLinks@2020-06-01' = {
  parent: privateDnsZone
  name: 'link-spoke-vnet'
  location: 'global'
  properties: {
    registrationEnabled: true // Auto-registers VMs
    virtualNetwork: {
      id:
'/subscriptions/66666666-7777-8888-9999-000000000000/resourceGroups/rg-network/providers/Micr
osoft.Network/virtualNetworks/vnet-spoke-prod'
    }
  }
}

// Link to Hub VNet (Resolution Only)
resource hubLink 'Microsoft.Network/privateDnsZones/virtualNetworkLinks@2020-06-01' = {
  parent: privateDnsZone
  name: 'link-hub-vnet'
  location: 'global'
  properties: {
    registrationEnabled: false // Resolution only
    virtualNetwork: {
      id:
'/subscriptions/66666666-7777-8888-9999-000000000000/resourceGroups/rg-network/providers/Micr
osoft.Network/virtualNetworks/vnet-hub-prod'
    }
  }
}
```

Q13. Deep dive into the Azure Monitor Agent (AMA) and Data Collection Rules (DCR). How does AMA replace the legacy Log Analytics Agent (MMA/OMS), and how do you configure a DCR to collect syslog and performance metrics?

Detailed Answer:

The Azure Monitor Agent (AMA) replaces the legacy Log Analytics Agent (MMA/OMS) and Diagnostic Extension. AMA improves security by using Microsoft Entra Managed Identities (system-assigned or user-assigned) instead of shared workspace keys.

The core architectural shift is the introduction of Data Collection Rules (DCRs). In the legacy model, configurations were set globally at the Log Analytics Workspace level, sending all data from all connected VMs to one workspace. With DCRs, you define *what* data to collect (e.g., specific event IDs, syslog facilities, performance counters) and *where* to send it (multiple Log Analytics workspaces, Azure Monitor Metrics, or Event Hubs). You then associate this DCR to specific VMs or VMSS instances. This allows granular, multi-tenant data routing and reduces data ingestion costs by filtering out noisy, unnecessary logs at the source.

Production Scenario / Practical Example:

The following Bicep configuration deploys a Data Collection Rule (DCR) that collects performance counters (CPU, Memory) and specific Syslog facilities (auth, mail), sending them to a central Log Analytics Workspace.

```
resource workspace 'Microsoft.OperationalInsights/workspaces@2022-10-01' existing = {
  name: 'law-central-ops'
  scope: resourceGroup('rg-mgmt-prod')
}

resource syslogDcr 'Microsoft.Insights/dataCollectionRules@2022-06-01' = {
  name: 'dcr-syslog-perf-linux'
  location: resourceGroup().location
  properties: {
    dataSources: {
      performanceCounters: [
        {
          streams: [ 'Microsoft-Perf' ]
          samplingFrequencyInSeconds: 10
          counterSpecifiers: [
            '\\Processor(_Total)\\% Processor Time'
            '\\Memory\\Available MBytes'
          ]
          name: 'perfCounterDataSource'
        }
      ]
      syslog: [
        {
          streams: [ 'Microsoft-Syslog' ]
          facilityNames: [ 'auth', 'authpriv', 'cron', 'mail' ]
          logLevels: [ 'Warning', 'Error', 'Critical', 'Alert', 'Emergency' ]
```

```
          name: 'syslogDataSource'
        }
      ]
    }
    destinations: {
      logAnalytics: [
        {
          workspaceResourceId: workspace.id
          name: 'centralWorkspace'
        }
      ]
    }
    dataFlows: [
      {
        streams: [ 'Microsoft-Perf', 'Microsoft-Syslog' ]
        destinations: [ 'centralWorkspace' ]
      }
    ]
  }
}
```

Q14. What are Azure Resource Locks? Explain the difference between `CanNotDelete` and `ReadOnly`, how inheritance behaves, and how an SRE can bypass or delete locks programmatically using Azure CLI and Terraform.

Detailed Answer:

Azure Resource Locks are control-plane mechanisms that prevent accidental deletion or modification of critical resources, regardless of the user's RBAC permissions (even subscription Owners are restricted by locks).

- **CanNotDelete (Delete)**: Authorized users can read and modify resources, but cannot delete them.
- **ReadOnly (ReadOnly)**: Authorized users can read resources, but cannot delete or modify them. *Note*: ReadOnly locks can cause unexpected failures for resources that require write-back operations during standard read actions (e.g., running a VM requires writing temporary state; a ReadOnly lock on a VM or its Resource Group will block it from starting).

Inheritance: Locks apply at the scope they are defined and are transitively inherited by all child resources. A lock applied at a Resource Group applies to all resources within it.

Bypassing / Deleting Locks: You cannot bypass a lock directly. To modify or delete a locked resource, an administrative identity with the `Microsoft.Authorization/locks/*` permission must first explicitly delete the lock, perform the resource operation, and then recreate the lock.

Production Scenario / Practical Example:

In a CI/CD or Terraform pipeline, locks must be managed carefully. Here is how an SRE programmatically

checks for, removes, and recreates a lock during a deployment script using Azure CLI.

```
RG_NAME="rg-core-networking"
LOCK_NAME="lock-prevent-deletion"

# Check if a lock exists
LOCK_EXISTS=$(az lock list --resource-group "$RG_NAME" --query "[?name=='$LOCK_NAME'].id" -o
tsv)

if [ -n "$LOCK_EXISTS" ]; then
    echo "Lock detected. Removing lock to allow deployment updates..."
    az lock delete --name "$LOCK_NAME" --resource-group "$RG_NAME"
fi

# Run Deployment / Updates (e.g., Terraform Apply or ARM deploy)
az group update --name "$RG_NAME" --tags "LastDeployed=$(date +%Y%m%d)"

# Recreate the CanNotDelete lock to preserve governance
echo "Recreating CanNotDelete lock on Resource Group..."
az lock create \
  --name "$LOCK_NAME" \
  --resource-group "$RG_NAME" \
  --lock-type CanNotDelete \
  --notes "Enforced by SRE platform governance team."
```

Q15. Explain Azure Compute Gallery (formerly Shared Image Gallery). How does it handle image versioning, cross-region replication, and how do you deploy a VMSS from a specific gallery image version?

Detailed Answer:

Azure Compute Gallery (ACG) is a service that helps you build structure and organization around your custom virtual machine images. It enables managing, versioning, and sharing images across subscriptions and Entra ID tenants.

Key Components:

1. Gallery: The logical container for managing images.

2. Image Definition: Defines the characteristics of the image (OS type: Linux/Windows, generation: Gen1/Gen2, VM SKU, and publishing state).

3. Image Version: The actual bootable VHD image. Versions follow semantic formatting (e.g., `1.0.0`, `2024.11.12`).

Replication and Scaling:

ACG allows you to replicate image versions to multiple target Azure regions. When deploying large-scale VMSS (e.g., hundreds of instances), a single storage account source can cause throttling. ACG solves this by letting you specify the number of replica counts per region. Each replica is an independent copy of the image, distributing the read load during massive scale-out events.

Production Scenario / Practical Example:

This Bicep snippet defines a VMSS deployment that references a specific image version from an Azure Compute Gallery.

```
param location string = resourceGroup().location
param vmssName string = 'vmss-app-workers'

resource vmss 'Microsoft.Compute/virtualMachineScaleSets@2023-09-01' = {
  name: vmssName
  location: location
  sku: {
    name: 'Standard_D2s_v5'
    tier: 'Standard'
    capacity: 5
  }
  properties
: {
    overprovision: false
    upgradePolicy: {
      mode: 'Automatic'
    }
    virtualMachineProfile: {
      storageProfile: {
        imageReference: {
          // Referencing the Azure Compute Gallery Image Version
          id:
'/subscriptions/66666666-7777-8888-9999-000000000000/resourceGroups/rg-shared-images/providers/Microsoft.Compute/galleries/gal_corp_images/images/ubuntu-hardened/versions/2.1.0'
        }
        osDisk: {
          caching: 'ReadWrite'
          createOption: 'FromImage'
          managedDisk: {
            storageAccountType: 'Premium_LRS'
          }
        }
      }
      osProfile: {
        computerNamePrefix: 'worker'
        adminUsername: 'sreadmin'
        linuxConfiguration: {
          disablePasswordAuthentication: true
          ssh: {
```

```
            publicKeys: [
              {
                path: '/home/sreadmin/.ssh/authorized_keys'
                keyData: 'ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQC...'
              }
            ]
          }
        }
      }
    }
  }
}
```

Q16. Detail the architecture of Azure Application Gateway v2. How do multi-site listeners, path-based routing rules, backend pools, and custom health probes function together?

Detailed Answer:

Azure Application Gateway v2 is a regional Layer 7 load balancer that supports SSL/TLS termination, Web Application Firewall (WAF), and URL-based routing.

Core Components & Flow:

1. Frontend IP: The ingress endpoint (can be public, private, or both).
2. Listeners: Evaluate incoming traffic.
   - *Basic Listener*: Listens on a single port (e.g., 80 or 443) for any domain.
   - *Multi-site Listener*: Listens on a specific hostname (e.g., `api.domain.com` and `app.domain.com`) using SNI (Server Name Indication) headers.
3. Routing Rules: Bind listeners to backend pools.
   - *Basic Routing*: Directs all traffic from a listener to a single backend pool.
   - *Path-based Routing*: Directs traffic to different backend pools based on the URL path (e.g., `/images/*` goes to Storage, `/api/*` goes to AKS).
4. Backend HTTP Settings: Define protocol (HTTP/HTTPS), port, cookie-based affinity, and connection draining timeout.
5. Custom Health Probes: Periodically test backend server health. A backend server is only marked healthy if it responds with a specific status code (e.g., `200-399`) within a configured timeout limit.
6. Backend Pools: The target compute resources (VMs, VMSS, App Services, or Private IPs).

Production Scenario / Practical Example:

An Azure CLI configuration establishing a path-based routing rule on an existing Application Gateway to split traffic between a legacy VM pool and an AKS cluster.

```
# Create path-based rule mapping /api/* to AKS, and default traffic to Legacy VMs
az network application-gateway url-path-map create \
  --gateway-name appgw-prod-01 \
  --resource-group rg-network-prod \
  --name route-map-01 \
  --paths "/api/*" \
  --address-pool aks-backend-pool \
  --http-settings aks-http-settings \
  --default-address-pool vm-backend-pool \
  --default-http-settings vm-http-settings

# Associate the path-based map with a routing rule
az network application-gateway rule create \
  --gateway-name appgw-prod-01 \
  --resource-group rg-network-prod \
  --name path-routing-rule \
  --address-pool vm-backend-pool \
  --http-settings vm-http-settings \
  --url-path-map route-map-01 \
  --listener appgw-https-listener
```

Q17. Compare Azure Public IP SKUs: Basic vs. Standard. What are the key architectural differences regarding security, availability zones, and dynamic vs. static allocation?

Detailed Answer:

Azure Public IPs are available in two SKUs: Basic and Standard. Understanding the differences is critical for designing secure, highly available architectures.

| Feature | Basic SKU (Legacy) | Standard SKU (Recommended) |

| :--- | :--- | :--- |

| Security (NSG requirement) | Open by default. No NSG is required to allow inbound traffic (highly insecure). | Closed by default. Inbound traffic is blocked unless explicitly permitted by an NSG. |

| Allocation Method | Dynamic (IP changes when resource is deallocated) or Static. | Static only (IP remains constant for the lifetime of the resource). |

| Availability Zones | Not zone-redundant. No support for zonal pinning. | Zone-redundant by default, or can be pinned to a specific Availability Zone (1, 2, or 3). |

| Routing Preference | Supports Microsoft Network routing only. | Supports Microsoft Network *and* Internet (Cold Potato) routing options. |

| Upgrade Path | Can be upgraded to Standard, but requires resource downtime. | Cannot be downgraded. |

For any production workload, Standard SKU is mandatory because it integrates with Standard Load Balancers,

NAT Gateways, and Azure Firewall, while supporting Zone Redundancy.

Production Scenario / Practical Example:

This Bicep snippet provisions a Standard, Zone-Redundant Static Public IP address suitable for an enterprise-grade Application Gateway or Firewall.

```
resource standardPublicIP 'Microsoft.Network/publicIPAddresses@2023-09-01' = {
  name: 'pip-appgw-prod-eastus2'
  location: 'eastus2'
  sku: {
    name: 'Standard'
    tier: 'Regional'
  }
  zones: [
    '1'
    '2'
    '3' // Zone-redundant deployment across all three zones
  ]
  properties: {
    publicIPAllocationMethod: 'Static' // Mandatory for Standard SKU
    publicIPAddressVersion: 'IPv4'
    idleTimeoutInMinutes: 4
  }
}
```

Q18. Compare Azure Service Endpoints and Private Endpoints. What are the architectural trade-offs regarding cost, routing, and access from on-premises environments?

Detailed Answer:

Choosing between Service Endpoints and Private Endpoints is a fundamental architectural decision that impacts network topology, security, and cost.

1. Service Endpoints

- **Routing**: Traffic leaves the VNet but remains on the Microsoft backbone. The target PaaS service resolves to its public IP address.
- **On-Premises Access**: Cannot be accessed directly from on-premises networks via ExpressRoute or VPN. On-premises traffic must use NAT to egress to the internet and access the public endpoint.
- **Cost**: Free of charge.
- **Complexity**: Low. No DNS modifications or dedicated IP addresses are required.
- **Granularity**: Coarse-grained. You allow access to the entire PaaS service type (e.g., all Azure SQL databases in the region) rather than a specific instance, which introduces data exfiltration risks.

2. Private Endpoints

- **Routing**: Traffic remains entirely private. The PaaS service is assigned a private IP address from your VNet.
- **On-Premises Access**: Fully accessible over ExpressRoute or VPN using the private IP.
- **Cost**: Incurs hourly runtime charges per Private Endpoint, plus data processing fees (per GB) for inbound/outbound traffic.
- **Complexity**: Medium-High. Requires Private DNS Zones, DNS forwarders, and IP address management.
- **Granularity**: Fine-grained. Access is restricted to a specific resource instance (e.g., `sql-prod-db-01`). This eliminates data exfiltration risks since traffic to other instances is blocked.

Production Scenario / Practical Example:

An architectural decision record (ADR) dictates using Private Endpoints for database engines (to prevent data exfiltration and allow on-premises DBA access) while using Service Endpoints for storage accounts containing ephemeral logs (to optimize data transfer costs).

```
# Example: Enabling Service Endpoints for Storage on an App Subnet
az network vnet subnet update \
  --resource-group rg-network-prod \
  --vnet-name vnet-spoke-prod \
  --name sn-app-tier \
  --service-endpoints Microsoft.Storage

# Example: Verifying that Private Endpoint IP is allocated in the subnet for SQL
az network private-endpoint show \
  --name pe-sql-prod \
  --resource-group rg-database-prod \
  --query "customDnsConfigs[0].ipAddresses[0]" -o tsv
# Output returns a private VNet IP: e.g., 10.0.2.15
```

Q19. Explain how to configure Azure ExpressRoute with VPN Failover. What routing protocols are used, and how do you manipulate BGP paths to ensure ExpressRoute is preferred?

Detailed Answer:

For enterprise connectivity, designing a resilient hybrid network requires a primary connection via Azure ExpressRoute and a secondary path via a Site-to-Site (S2S) VPN as a backup.

Routing Mechanics (BGP):

Both ExpressRoute and S2S VPN connections use Border Gateway Protocol (BGP) to exchange routes between on-premises and Azure VNets. When both paths are active, Azure prefers the ExpressRoute path over the S2S VPN because ExpressRoute routes have a shorter path metric (AS Path) and higher priority by default.

To configure active-backup routing:

1. ExpressRoute Gateway: Deploy an ExpressRoute Gateway in the Hub VNet.

2. VPN Gateway: Deploy a VPN Gateway (Route-Based) in the same Hub VNet.

3. AS Path Prepending: On-premises routers should advertise routes to Azure via both paths. To ensure ExpressRoute is preferred for outbound traffic from on-premises to Azure, advertise routes via VPN with a longer AS Path (prepend your local ASN multiple times).

4. Local Preference: On-premises routers should set a higher Local Preference for routes received from Azure via ExpressRoute.

Production Scenario / Practical Example:

Below is a conceptual Cisco IOS configuration snippet representing the on-premises router's BGP policy, prepending AS paths on the backup VPN connection to enforce ExpressRoute preference.

```
! On-Premises Router BGP Configuration
router bgp 65001
 neighbor 192.168.10.1 remote-as 12076  ! ExpressRoute Peer (Azure)
 neighbor 192.168.20.1 remote-as 65515  ! VPN Peer (Azure)
 !
 address-family ipv4
  neighbor 192.168.10.1 route-map ER_INBOUND in
  neighbor 192.168.20.1 route-map VPN_OUTBOUND out
 exit-address-family
!
! Prefer ExpressRoute inbound by setting high local preference
route-map ER_INBOUND permit 10
 set local-preference 200
!
! Prepend AS Path on VPN outbound to make it less attractive to Azure
route-map VPN_OUTBOUND permit 10
 set as-path prepend 65001 65001 65001
```

Q20. Explain Azure Managed Identities. How do System-Assigned and User-Assigned identities differ under the hood, and how does a VM fetch an access token from the Instance Metadata Service (IMDS)?

Detailed Answer:

Managed Identities eliminate the need for developers to manage credentials (connection strings, certificates, secrets) in code. They are wrappers around service principals in Microsoft Entra ID.

- **System-Assigned Managed Identity**:
- Tied directly to the lifecycle of a single Azure resource (e.g., a VM or App Service).

- When the resource is deleted, Azure automatically cleans up the identity in Entra ID.
- Shared 1:1 relationship.
- **User-Assigned Managed Identity**:
- Created as a standalone Azure resource.
- Can be assigned to multiple Azure resources (1-to-many relationship).
- Useful for workloads where multiple resources need identical permissions (e.g., a cluster of VMs).
- Its lifecycle is managed independently.

Under the Hood (IMDS Mechanics):

When a resource (like a VM) requests a token, it queries the local non-routable IP address `169.254.169.254` (Azure Instance Metadata Service - IMDS) via HTTP over port 80. The request must include the header `Metadata: true` to prevent Server-Side Request Forgery (SSRF) attacks.

Production Scenario / Practical Example:

An SRE logs into a private Linux VM and executes a curl command to programmatically fetch an access token for Azure Resource Manager from the IMDS endpoint, then uses it to query subscription details.

```
# 1. Fetch the access token from the IMDS endpoint
# Note: 'resource' parameter specifies the target service (ARM API)
TOKEN_RESPONSE=$(curl -s -H "Metadata: true" \

"http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resource=https://management.azure.com/")

# Extract the access token using jq
ACCESS_TOKEN=$(echo "$TOKEN_RESPONSE" | jq -r .access_token)

# 2. Use the token to securely query the Azure Resource Manager REST API
curl -s -H "Authorization: Bearer $ACCESS_TOKEN" \
  "https://management.azure.com/subscriptions?api-version=2020-01-01" | jq .
```